Benchmarking Sample Efficiency: Synthetic Data Augmentation vs. Latent World Models in Continuous Control

Tao He

Northeastern University
Boston, MA

Abstract

Sample efficiency is a central bottleneck for applying reinforcement learning (RL) to continuous control. Two broad families of methods aim to address it: model-based RL with synthetic data augmentation (e.g., MBPO), and latent world models (e.g., Dreamer), which learn a compact latent representation of the environment. Despite growing empirical interest in both directions, their relative strengths on low-dimensional, state-vector continuous control tasks—where the "world model" no longer benefits from rich visual encoders—are not well understood. In this paper, we benchmark four algorithms (SAC, MBPO, a verified MLP-encoder variant of Dreamer, and a Stable-Baselines3 PPO baseline) on four environments of increasing complexity: Pendulum-v1, LunarLanderContinuous-v3, InvertedDoublePendulum-v4, and HalfCheetah-v4. For each (algorithm, environment) pair we aggregate over multiple independent seeds and report both sample steps to a predefined convergence threshold and asymptotic performance. We find that (i) off-policy SAC and synthetic-data-augmentation MBPO are strong and reach their thresholds comparably fast on several low-dimensional tasks, while SAC additionally attains the highest asymptotic return on HalfCheetah; (ii) MBPO's advantage diminishes as imagination horizon grows on the high-dimensional task, exposing compounding model error; and (iii) the MLP Dreamer, despite its theoretical appeal, fails on both LunarLander and InvertedDoublePendulum, which we attribute to hard contact discontinuities and very short episodes interacting poorly with a non-visual RSSM and burn-in initialization. Ablations further show that λ-returns and the symlog reward/value transformation are both necessary for Dreamer stability, but are not sufficient to overcome these architectural limitations. Source code is available at https://github.com/taohecs-code/continuous_control_sac_mbpo_mlpdreamer.

Introduction

Reinforcement Learning (RL) relies heavily on trial-and-error interactions. However, collecting enough data is often difficult in real-world applications due to high costs or safety concerns. Sample efficiency measures how well an algorithm uses data, specifically referring to how much interaction an agent needs to reach a certain performance level. While standard RL works well in discrete environments, real-world applications usually involve continuous control tasks. In tasks like robotic manipulation or autonomous driving, the state and action spaces are infinite. In these cases, exploring the environment is expensive, and the amount of data needed grows exponentially. Therefore, sample efficiency is very important for practical deployment.

To improve sample efficiency, researchers usually take three main paths. The first path focuses on improving the base algorithms to learn better from incoming data. The second path uses off-policy methods, like replay buffers, to reuse past experiences. Third, Model-Based Reinforcement Learning (MBRL) learns a model of the environment to simulate interactions. Recently, latent world models have become popular in MBRL because they can learn good representations of the environment, which reduces the need for real-world interactions.

This paper evaluates three different RL methods: SAC (model-free), MBPO (model-based), and Dreamer (world models). We test them on various continuous control tasks to compare their sample efficiency. We also explain why they perform differently across tasks, ranging from simple low-dimensional ones (like LunarLanderContinuous) to complex MuJoCo locomotion tasks.

Background

RL is based on an agent interacting with an environment. The agent sees a state s, takes an action a, and gets a reward r. Early RL used tabular methods to record values for each state–action pair, typically using greedy or ϵ-greedy strategies. However, tabular methods fail in continuous environments because the state space is too large.

To handle continuous states, value-based methods like Deep Q-Networks (DQN) (Mnih et al. 2015) use neural networks to approximate values. While DQN handles infinite states, it only works for discrete actions because finding the maximum Q-value in a continuous action space is too hard to compute.

Policy Gradient (PG) methods solve the continuous action problem by directly optimizing the policy. Early algorithms like REINFORCE (Williams 1992) used Monte Carlo sampling but suffered from high variance. The Actor-Critic (AC) architecture combines PG with Temporal Difference (TD) learning. The Actor chooses actions, and the Critic estimates the value to evaluate those actions, which lowers the

Copyright © 2026, Association for the Advancement of Artificial Intelligence (www.aaai.org). All rights reserved.

variance.

Early continuous control algorithms like DDPG used deterministic policies and added noise for exploration. Modern algorithms usually prefer stochastic policies where the Actor outputs a probability distribution. To keep the network differentiable, they use the reparameterization trick. This separates the noise from the network parameters so gradients can flow through the sampled actions:

$$a = \mu(s) + \sigma(s) \odot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I), \tag{1}$$

where $\mu(s)$ and $\sigma(s)$ are the mean and (elementwise) standard deviation of the policy head; the specific parameterization (ϕ for SAC, ψ for Dreamer) is introduced alongside each algorithm below.

Related Work

In model-free RL, experience replay is a standard way to improve sample efficiency. The replay buffer stores past transitions, allowing the agent to reuse historical data. DDPG (Lillicrap et al. 2016) is an early example, though it suffers from instability and overestimation because of its deterministic feature. Later algorithms like TD3 (Fujimoto, van Hoof, and Meger 2018) and SAC (Haarnoja et al. 2018a) fixed these issues using techniques like clipped double Q-learning. SAC also improved exploration by using a maximum entropy framework.

The idea of combining learning and planning with an environment model started with the Dyna architecture (Sutton 1990). Based on this, several MBRL algorithms were developed. PILCO (Deisenroth and Rasmussen 2011) showed great sample efficiency using Gaussian Processes, but it is hard to scale to high-dimensional spaces because it is computationally expensive. PETS (Chua et al. 2018) introduced probabilistic neural network ensembles to handle uncertainty, but its online planning is still slow. ME-TRPO (Kurutach et al. 2018) updated Dyna by training model-free policies using simulated rollouts from learned models. MBPO (Janner et al. 2019) combined these ideas, using ensemble models and short-horizon rollouts to prevent model errors from accumulating.

Another approach is learning latent dynamics, often called World Models. Ha and Schmidhuber (2018) showed that agents can learn policies entirely inside a simulated latent space using a VAE and an RNN. PlaNet (Hafner et al. 2019) introduced the Recurrent State-Space Model (RSSM) for latent planning, separating deterministic memory from stochastic states. However, PlaNet used the Cross-Entropy Method (CEM) for online planning, which is slow. Dreamer (Hafner et al. 2020) solved this by training an actor–critic policy directly in the latent space. Later Dreamer versions added categorical variables (Hafner et al. 2021) and larger-scale architectures (Hafner et al. 2023) to handle harder tasks.

Methodology

Soft Actor-Critic (SAC)

SAC is an off-policy, model-free RL algorithm based on the actor–critic architecture (Haarnoja et al. 2018a). It adds an entropy term to the actor's objective to encourage exploration:

$$J(\pi) = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t \big(r(s_t, a_t) + \alpha\, \mathcal{H}(\pi(\cdot|s_t)) \big) \right] \tag{2}$$

where the expectation is taken over trajectories induced by π, t denotes the environment time step, $\gamma \in [0, 1)$ is the discount factor, and $\mathcal{H}$ is the policy entropy:

$$\mathcal{H}(\pi(\cdot|s_t)) = \mathbb{E}_{a_t \sim \pi}[-\log \pi(a_t|s_t)]\,, \tag{3}$$

The coefficient α is the temperature that controls how strongly the entropy term is weighted against reward. Because our actor is a $\tanh$-squashed Gaussian (see Eq. (6) below), the analytical form of $\mathcal{H}$ is intractable; we therefore estimate it by a single-sample Monte-Carlo estimator $\widehat{\mathcal{H}}(s_t) = -\log \pi_\phi(a_t|s_t)$ at the reparameterized action a_t (with the usual $\tanh$-correction applied to $\log \pi_\phi$), which is why the actor loss Eq. (7) contains $\alpha \log \pi_\phi(a_t|s_t)$ rather than $-\alpha\mathcal{H}$.

To reduce overestimation bias, SAC uses two Q-networks, Q_{θ_1} and Q_{θ_2}, and takes the minimum for the target value. The critics minimize the mean squared error:

$$J_Q(\theta_i) = \mathbb{E}_{(s_t,a_t)\sim\mathcal{D}} \left[\tfrac{1}{2} \big(Q_{\theta_i}(s_t, a_t) - y \big)^2 \right],\ i \in \{1, 2\} \tag{4}$$

where the target y is

$$\begin{aligned} y = r(s_t, a_t) + \gamma\, \mathbb{E}_{a_{t+1}\sim\pi_\phi} \Big[& \min_{j=1,2} Q_{\theta'_j}(s_{t+1}, a_{t+1}) \\ & - \alpha\, \log \pi_\phi(a_{t+1}|s_{t+1}) \Big]. \end{aligned} \tag{5}$$

The target-critic parameters θ'_j are updated by Polyak averaging with a scalar coefficient $\tau_\mathrm{p} \in (0, 1]$ (we use $\tau_\mathrm{p} = 0.005$):

$$\theta'_j \leftarrow \tau_\mathrm{p}\, \theta_j + (1 - \tau_\mathrm{p})\, \theta'_j.$$

The actor is a $\tanh$-squashed Gaussian: at each state s_t it outputs $\mu_\phi(s_t)$ and $\sigma_\phi(s_t)$, and an action is sampled via the reparameterization trick

$$a_t = \tanh\big(\mu_\phi(s_t) + \epsilon \odot \sigma_\phi(s_t)\big),\ \epsilon \sim \mathcal{N}(0, I), \tag{6}$$

where ϵ is standard-normal reparameterization noise drawn independently for each transition and $\odot$ is element-wise product. The policy objective is

$$J_\pi(\phi) = \mathbb{E}_{s_t,\,\epsilon} \left[\alpha \log \pi_\phi(a_t|s_t) - \min_j Q_{\theta_j}(s_t, a_t) \right]. \tag{7}$$

In our implementation, we address the instability of auto-entropy tuning. Manually tuning α is hard because the best entropy scale changes during training. While SAC can automatically adjust α to match a target entropy $\bar{\mathcal{H}}$, it can still cause boundary issues. The standard loss for α is

$$\mathcal{L}(\alpha) = -\mathbb{E}_{a_t\sim\pi_t} \big[\alpha \big(\log \pi_t(a_t|s_t) + \bar{\mathcal{H}} \big) \big]\,. \tag{8}$$

To prevent α from becoming negative, our implementation optimizes its logarithm ($\log \alpha$) instead:

$$\mathcal{L}(\log \alpha) = -\mathbb{E}_{a_t\sim\pi_t} \big[\log \alpha \big(\log \pi_t(a_t|s_t) + \bar{\mathcal{H}} \big) \big]\,. \tag{9}$$

Model-Based Policy Optimization (MBPO)

MBPO combines the sample efficiency of model-based methods with the performance of model-free SAC. It alternates between learning an environment model and using it to generate synthetic data.

MBPO learns an ensemble of E probabilistic networks $\{\theta_1, \ldots, \theta_E\}$. Each network predicts the joint target $(\Delta s_t, r_t)$ — the residual state change $\Delta s_t = s_{t+1} - s_t$ together with the reward — as a Gaussian distribution:

$$p_{\theta_i}\big(\Delta s_t,\, r_t \,\big|\, s_t, a_t\big) = \mathcal{N}\big(\mu_{\theta_i}(s_t, a_t),\, \Sigma_{\theta_i}(s_t, a_t)\big). \quad (10)$$

At rollout time, the next state is reconstructed as $s_{t+1} = s_t + \Delta s_t$.

The ensemble is trained on real transitions from the environment buffer $\mathcal{D}_{\text{env}}$. To prevent overfitting, the data is split into a training set $\mathcal{D}_{\text{train}}$ and a validation set $\mathcal{D}_{\text{val}}$. The models minimize the negative log-likelihood (NLL):

$$\mathcal{L}_{\text{model}}(\theta_i) = -\mathbb{E}_{\mathcal{D}_{\text{train}}}\big[\log p_{\theta_i}\big(\Delta s_t,\, r_t \,\big|\, s_t, a_t\big)\big]\,. \quad (11)$$

When generating synthetic data, the algorithm selects the top K models based on their validation MSE:

$$\text{MSE}_{\text{val}}(\theta_i) = \mathbb{E}_{\mathcal{D}_{\text{val}}}\Big[\big\|\mu_{\theta_i}(s_t, a_t) - (\Delta s_t,\, r_t)\big\|^2\Big]\,. \quad (12)$$

The synthetic transitions are stored in a model buffer $\mathcal{D}_{\text{model}}$. The SAC agent is then updated using a mix of real and synthetic data.

In our implementation, we made a modification to the standard MBPO. While the original paper uses a dynamic schedule for the rollout horizon, we use a fixed horizon h and a fixed number of model updates per step. This allows us to conduct clean ablation studies to see how the imagination depth affects sample efficiency and compounding errors (detailed in the Experiments section).

Verified Dreamer

RSSM Dynamics Model. Dreamer is a model-based RL algorithm based on the Recurrent State-Space Model (RSSM). RSSM splits the latent state into a deterministic part h_t and a stochastic part z_t. h_t is implemented with an RNN to keep historical context, while z_t is sampled to capture environment randomness. The latent space has a "prior" predictor and a "posterior" representation. The prior predicts the next state using only history h_t. The posterior corrects this belief using the current encoded observation e_t. The dynamics are

$$h_t = f_\theta(h_{t-1}, z_{t-1}, a_{t-1}), \quad (13)$$

$$\hat{z}_t \sim p_\theta(\hat{z}_t \mid h_t), \quad (14)$$

$$z_t \sim q_\phi(z_t \mid h_t, e_t), \quad (15)$$

where e_t represents the encoded observation.

During real environment interaction, the agent uses actual observations to choose actions and collect data. This data is stored in a replay buffer to train the World Model. After the World Model is updated, the Actor and Critic are trained entirely on imagined trajectories generated by the World Model.

To generate accurate rollouts, Dreamer uses a "burn-in" technique. The World Model processes a sequence of real past observations to initialize its latent state. However, for environments with short episodes like InvertedDoublePendulum (IDP), a fixed burn-in sequence (e.g., 50 steps) might cross episode boundaries and mix unrelated states. To fix this, our replay buffer dynamically adjusts the burn-in length to the longest valid window without an end-of-episode signal.

To train the RSSM, the model maximizes the Evidence Lower Bound (ELBO). The reconstruction losses for observations and rewards represent the data likelihood $\log P(\text{obs}) + \log P(\text{reward})$, so their theoretical weight is 1.0. The KL divergence between the posterior q_ϕ and the prior p_θ is a regularization term. Similar to β-VAE (Higgins et al. 2017), a coefficient β_{KL} is added to weight the KL divergence and prevent posterior collapse. Predicting episode continuation is also a data likelihood, but to handle sparse death signals in some environments, we use β_{cont} to scale the binary cross-entropy loss.

To prevent the prior from dominating the posterior, Dreamer uses KL Balancing. It uses a stop-gradient operator $\text{sg}(\cdot)$ to split the KL into two terms with different weights: a prior-update term $\text{KL}\big(\text{sg}(q_\phi) \,\|\, p_\theta\big)$, whose gradient flows only through the prior p_θ and trains the prior to match the (frozen) posterior, and a posterior-regularization term $\text{KL}\big(q_\phi \,\|\, \text{sg}(p_\theta)\big)$, whose gradient flows only through the posterior q_ϕ and regularizes it toward the (frozen) prior. A Free Nats threshold is used to prevent over-regularization: only KL values above this threshold are penalized. Finally, an Auto KL Beta mechanism adjusts β_{KL} to track a target KL. The KL loss is

$$\mathcal{L}_{\text{KL}} = \alpha_{\text{bal}}\, \text{KL}\big(\text{sg}(q_\phi) \,\|\, p_\theta\big) + (1 - \alpha_{\text{bal}})\, \text{KL}\big(q_\phi \,\|\, \text{sg}(p_\theta)\big), \quad (16)$$

where $\alpha_{\text{bal}} \in [0, 1]$ is the KL-balancing weight (we use $\alpha_{\text{bal}} = 0.8$, following DreamerV2), which is distinct from the SAC entropy temperature α used earlier.

While Dreamer is often used for pixel-based environments with CNNs, we evaluate it on low-dimensional state-vector inputs. We use a simple Multilayer Perceptron (MLP) for the observation encoder. The MLP encoder is not pretrained separately; the entire World Model (encoder, RSSM, decoders) is trained end-to-end. The overall loss is

$$\mathcal{L}_{\text{WM}} = \mathcal{L}_{\text{obs}} + \mathcal{L}_{\text{reward}} + \beta_{\text{cont}}\, \mathcal{L}_{\text{continue}} + \beta_{\text{KL}}\, \mathcal{L}_{\text{KL}}. \quad (17)$$

Actor and Critic in Dreamer. Dreamer trains the actor π_ψ and critic V_ξ on trajectories imagined by the RSSM; here ψ denotes the actor parameters and ξ denotes the critic parameters, both distinct from the world-model parameters θ, ϕ introduced above. The critic uses TD learning, computing λ-returns G_τ via backward recursion:

$$G_\tau = \hat{r}_\tau + \gamma\, \hat{c}_\tau [(1 - \lambda)\, V_{\xi'}(z_{\tau+1}) + \lambda\, G_{\tau+1}]\,, \quad (18)$$

where $\tau \in \{1, \ldots, H\}$ is the *imagined rollout step*, distinct both from the real environment time step t and from the SAC Polyak coefficient τ_{p} introduced earlier; $\hat{r}_\tau$ is the reward head's prediction for step τ and $\hat{c}_\tau \in [0, 1]$ is the continuation-probability head's prediction (i.e., the probability that the episode has not terminated by step τ). $V_{\xi'}$ is

the target critic: a copy of V_ξ whose weights ξ' are maintained by an exponential moving average of ξ, providing a stable anchor for value estimation beyond the imagined horizon.

Dreamer's actor update is different from model-free algorithms like SAC. In SAC, the actor relies on the live critic for gradients. In Dreamer, the actor gets gradients directly through the differentiable World Model over the imagined rollout. Because the actor already gets enough gradient signals from the World Model, the critic's job is just to provide a bootstrap value at the end of the rollout. Using a live critic here would cause the λ-returns to fluctuate and make the actor unstable. Therefore, Dreamer uses the stable target critic $V_{\xi'}$ for the actor's loss.

The actor loss also applies a per-step discount factor γ^τ along the imagined rollout. While PPO or A2C often drop this term, Dreamer needs it: the World Model is imperfect, and errors accumulate over τ. The γ^τ term reduces the weight of later, less reliable steps, preventing the actor from exploiting long-term hallucinations.

To keep actions within the valid range (e.g., $[-1, 1]$), the actor uses a $\tanh$ squashing function. Since we cannot easily compute the analytical entropy of a squashed Gaussian, we estimate it by sampling, using the negative log-probability $-\log \pi_\psi(a_\tau \mid z_\tau)$. This is added to the actor's objective for exploration. The final actor loss is

$$\mathcal{L}_{\text{Actor}} = -\mathbb{E}\left[\sum_{\tau=1}^{H} \gamma^\tau \big(G_\tau - \eta \log \pi_\psi(a_\tau \mid z_\tau)\big)\right], \quad (19)$$

where H is the imagination horizon and η is the entropy coefficient (analogous in role to the SAC temperature α, but a separate, manually tuned hyperparameter in our Dreamer implementation).

In environments with extreme rewards, the MSE loss can cause exploding gradients. DreamerV3 uses a symlog transformation to compress rewards and values:

$$\text{symlog}(x) = \text{sign}(x) \cdot \ln(1 + |x|), \quad (20)$$

$$\text{symexp}(x) = \text{sign}(x) \cdot (e^{|x|} - 1). \quad (21)$$

In our implementation, this is used in two places. First, during World Model training, true rewards are compressed with symlog before calculating the loss. Second, the critic predicts values in this compressed space.

However, calculating the λ-return requires addition, which is invalid in logarithmic space. To fix this, the predicted rewards and values are first decompressed to linear space using symexp. The λ-return is computed in this linear space, and the target return is compressed again using symlog before training the critic.

Experiments

Setup

We evaluate three representative algorithms alongside a robust baseline: SAC (off-policy model-free), MBPO (model-based), and Dreamer (latent world models). PPO (Proximal Policy Optimization) (Schulman et al. 2017) from the Stable-Baselines3 library serves as an on-policy model-free baseline. We designed a progressive benchmarking suite comprising four continuous control tasks with varying levels of complexity. To obtain statistically meaningful comparisons, each (algorithm, environment) pair is run with multiple independent random seeds, and all reported curves and numbers are aggregated (mean $\pm$ std) over these seeds — throughout the paper, shaded bands and "$\pm$" values denote one standard deviation across seeds: Pendulum-v1 uses 5 seeds per algorithm (seed 1–5), and LunarLanderContinuous-v3, InvertedDoublePendulum-v4, and HalfCheetah-v4 each use 3 seeds per algorithm (seed 1–3). Table 1 summarizes the specific details of these environments.

Environment	$\lvert\mathcal{S}\rvert$	$\lvert\mathcal{A}\rvert$	Range	Horizon	Thr.	Seeds
Pendulum-v1	3	1	$[-2, 2]$	200	-200	5
LunarLanderContinuous-v3	8	2	$[-1, 1]$	1000	200	3
InvertedDoublePendulum-v4	11	1	$[-1, 1]$	1000	9000	3
HalfCheetah-v4	17	6	$[-1, 1]$	1000	3000	3

Table 1: Evaluation environments and their predefined convergence thresholds. Horizon is the maximum episode length.

Instead of relying on arbitrary historical leaderboards, we define our evaluation thresholds based on official Gymnasium documentation and the specific goal of measuring early-stage sample efficiency. For LunarLanderContinuous-v3, we use the official *reward threshold* of 200 defined in the Gymnasium source code, which indicates the environment is considered "solved". For Pendulum-v1, while the environment does not define an explicit *reward threshold*, the theoretical maximum reward is 0 (perfectly upright with zero velocity and no torque). In a 200-step episode, based on our empirical learning curves, successful algorithms reliably plateau above -200. Therefore, we adopt -200 as a rigorous threshold indicating a successfully learned swing-up and balancing policy.

For the MuJoCo environments, we establish thresholds based on the environment's inherent reward structure and our restricted interaction budget. For InvertedDoublePendulum-v4, the maximum episode length is 1000 steps, and the agent receives a base reward of 10 per step minus penalty terms. Surviving the entire episode yields a theoretical maximum near 10,000. Based on our empirical learning curves, all baseline algorithms reliably plateau above 9000, making 9000 a rigorous threshold for convergence. For HalfCheetah-v4, reaching the absolute maximum asymptotic performance often requires millions of interaction steps (e.g., SAC reaches over 15,000 after 3M steps (Haarnoja et al. 2018b)). Because our study specifically benchmarks sample efficiency under a limited budget, we define a "preliminary convergence threshold" of 3000. This target represents the critical point where the agent transitions from random flailing to a stable, functional forward locomotion gait, allowing us to clearly compare how fast each algorithm learns in the early stages of training.

The primary evaluation metric is the *Sample Steps to Threshold*, defined as the number of environment interactions required for an algorithm to first reach the respective threshold and maintain above-threshold returns for three consecutive evaluation points. To ensure a strictly fair comparison, the total step count includes all initial random prefill (warm-up) steps. We additionally report the *Asymptotic Performance*, calculated as the mean $\pm$ std over seeds of each seed's average return across the final 10% of its evaluation episodes.

All environmental settings and reward systems are defined using the official Gymnasium API, and the algorithms are implemented using the PyTorch framework. To separate the effect of the algorithm from the network's capacity, the base MLP architectures are standardized across our custom implementations. With the exception of the Stable-Baselines3 PPO baseline, which retains its default library configuration, the actor and critic networks feature two hidden layers with identical dimensions and learning rates across all other methods. Common hyperparameters, such as the soft target update coefficient τ_{p}, are kept uniform. For stochastic policies, the log standard deviation is clamped within $[-20, 2]$ to ensure numerical stability. SAC, MBPO, and Dreamer use a squashed Gaussian policy with a $\tanh$ transformation.

Results and Analysis

Figure 1 shows the learning curves of the four algorithms across the benchmark environments. Table 2 summarizes, for each environment, the sample steps required to first reach the predefined convergence threshold (maintained for three consecutive evaluation points) and the final asymptotic performance. For each environment, the MBPO horizon reported is the one that achieved the highest peak evaluation return among available groups (Pendulum: $h = 15$; LunarLander: $h = 3$; IDP: $h = 1$; HalfCheetah: $h = 3$).

Low-Dimensional Continuous Control. In the Pendulum environment, SAC, MBPO, and Dreamer all converge. However, PPO fails to reach the -200 threshold within the limited steps. Because PPO is an on-policy algorithm, it discards data after each update, making it highly sample-inefficient compared to off-policy and model-based methods that reuse historical transitions.

High-Dimensional Locomotion. The HalfCheetah environment highlights the trade-off between early-stage sample efficiency and final asymptotic performance. MBPO shows exceptional early-stage efficiency and can shoot up to 3000 points much faster than SAC. This is because MBPO generates short-horizon synthetic data to multiply the number of policy updates per real environment step. However, MBPO plateaus at an asymptotic return of 6297 ± 1460 (Table 2). This plateau is caused by "model bias"—as the state space gets complex, the ensemble models inevitably make small errors, which limits the final policy quality.

In contrast, SAC learns slightly slower at the beginning but achieves the highest final asymptotic return (7825 ± 1342), as it relies entirely on ground-truth environment dynamics without model errors. Dreamer achieves a moderate asymptotic return of 3223 ± 1532 (peak near 5300). While

Env. (thr.)	Algorithm	Steps to Thr.	Asymptotic
Pendulum (-200)	SAC	4k (5/5)	-132.6 ± 10.2
	MBPO (h15)	3k (5/5)	-132.0 ± 10.2
	Dreamer	7k (5/5)	-142.7 ± 14.3
	SB3 PPO	– (0/5)	-1049.9 ± 103.7
LunarLander (200)	SAC	85k (2/3)	174.3 ± 100.6
	MBPO (h3)	– (0/3)	-13.0 ± 15.2
	Dreamer	– (0/3)	-340.7 ± 84.7
	SB3 PPO	229k (2/3)	142.3 ± 43.6
IDP (9000)	SAC	18k (3/3)	9357.9 ± 0.6
	MBPO (h1)	18k (3/3)	9358.0 ± 1.1
	Dreamer	70k (1/3)	2064.7 ± 2083.5
	SB3 PPO	166k (3/3)	6127.2 ± 4086.9
HalfCheetah (3000)	SAC	56k (3/3)	7824.8 ± 1341.8
	MBPO (h3)	93k (3/3)	6297.5 ± 1460.0
	Dreamer	145k (2/3)	3223.0 ± 1531.7
	SB3 PPO	659k (1/3)	2240.1 ± 958.8

Table 2: Main comparison on four continuous-control environments. "Steps to Thr." shows the average number of environment steps required to first reach the threshold and stay above it for three consecutive evaluations; in parentheses we report the number of seeds that ever reached the threshold. "–" denotes that the threshold was never met within the interaction budget. "Asymptotic" is the mean $\pm$ std across seeds of each seed's average return over its last 10% of evaluations. For MBPO we report, for each environment, the single horizon that achieved the highest peak return among the available horizon groups (Pendulum: $h = 15$; LunarLander: $h = 3$; IDP: $h = 1$; HalfCheetah: $h = 3$); the full horizon sweep is provided as an ablation in Table 3.

Dreamer is highly successful in pixel-based environments using CNNs, our simplified MLP-based World Model struggles to perfectly predict the 17-dimensional continuous kinematics, leading to compounding errors in the 15-step imagined rollouts.

The Failure Case: Discontinuous Dynamics. LunarLander presents the most striking failure case for Dreamer. As Table 2 shows, SAC and PPO partially solve the task — $2/3$ seeds each transiently cross the 200 threshold, with asymptotic returns of 174 ± 101 and 142 ± 44 respectively (the large seed-to-seed variance reflects occasional crashes even for the better methods). MBPO is already a failure case in its own right — none of its 3 seeds reach 200 and it plateaus near zero (-13 ± 15) — but Dreamer fails far more severely, collapsing to -341 ± 85, a regime where the agent is consistently crashing the lander rather than merely failing to land. We attribute this qualitative gap between Dreamer and the other three methods to the "hard contact dynamics" of the environment. When the lander's legs touch the ground, the physical forces and states change abruptly (a discontinuity). Simple MLP-based latent dynamics models struggle to predict these sharp, non-smooth transitions accurately, even with the symlog transformation. When the World Model makes a prediction error at the moment of impact, the error explodes over the subsequent imagined steps. Consequently,

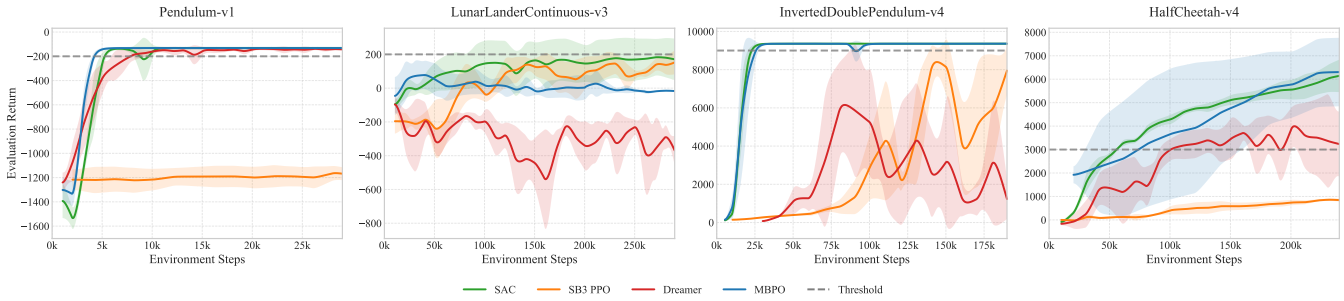

Figure 1: Learning curves (mean $\pm$ std, shaded) of SAC, MBPO, Dreamer, and SB3 PPO on the four benchmark environments. The dashed grey line marks the predefined convergence threshold for each task. For each environment, the x-axis is truncated at the shortest maximum step across algorithms to ensure a fair early-efficiency comparison.

the actor is trained on completely unrealistic hallucinated crashes, preventing it from learning a safe landing policy. This highlights a critical limitation of latent world models: they are highly sensitive to environments with non-smooth, discontinuous physical rules.

On InvertedDoublePendulum-v4, SAC and MBPO ($h = 1$) both reach the 9000 threshold at essentially the same point ($\approx$ 18k steps, all 3/3 seeds) and then plateau tightly around 9358, with the two methods' learning curves in Figure 1 almost indistinguishable. PPO is more subtle: as reported in Table 2, each of its 3 seeds individually crosses 9000 at some point (their *per-seed* first-crossing times average $\approx$ 166k steps, hence the 3/3 in the table), but the seeds cross at very different real times and none of them *stably* remains above 9000. As a result, the across-seed mean curve in Figure 1 (orange) never visually crosses the threshold and ends closer to 2000, and the final asymptotic return is only 6127 ± 4087 — the ± 4087 standard deviation is a direct indicator of this instability, showing that PPO's on-policy updates struggle to consistently exploit this narrow balancing task even when given far more environment interactions than SAC or MBPO. Dreamer is the only algorithm that largely fails on IDP: only $1/3$ seeds ever crosses 9000 (at $\approx$ 70k steps, and without stably remaining above it), and the cross-seed asymptotic return is 2064 ± 2084, one order of magnitude below the other three methods; the large per-seed spread (± 2084) visible as the wide red shaded band in Figure 1 reflects that the remaining two seeds plateau far below threshold. We believe this is a direct consequence of IDP's very short episode length (episodes terminate as soon as the pendulum falls, often within tens of steps early in training). A short episode leaves our replay buffer with very few transitions that can serve as a valid burn-in context for the RSSM, and the dynamically truncated burn-in cannot fully compensate: the recurrent module and the posterior rarely see sequences long enough to converge to a useful latent representation before the episode ends, so the imagined rollouts on which the actor is trained remain unreliable. Whether this failure is primarily due to an insufficient burn-in length, or due to the RNN's recurrent state itself not converging on such short sequences, cannot be fully disentangled with our current data; we return to this in the *Future Work* section, where we outline shorter-burn-in and zero-initialization variants that would isolate the two causes.

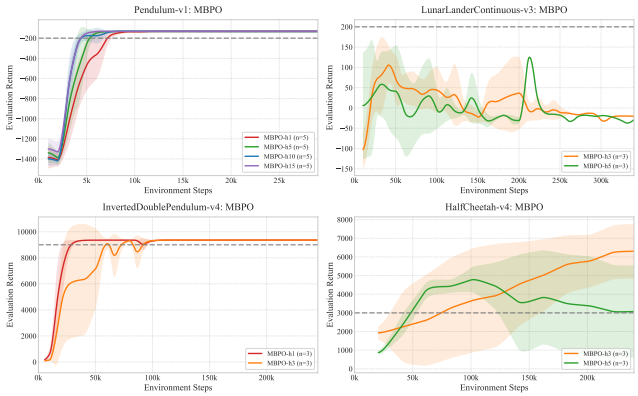

Figure 2: MBPO imagination-horizon ablation across the four environments (mean $\pm$ std, shaded). The dashed grey line indicates the convergence threshold.

Ablation Study

We conduct two sets of ablations. The first sweeps the MBPO imagination horizon h across all four environments to study the trade-off between the amount of synthetic data and compounding model error. The second isolates two key design choices inside our Dreamer implementation—the use of λ-returns (versus a 1-step TD(0) target) and the symlog reward/value transformation—on the two environments where they are most informative (LunarLander and InvertedDoublePendulum).

MBPO Horizon Ablation. Figure 2 shows the learning curves for each horizon setting per environment, and Table 3 reports the corresponding quantitative summary.

For the simple Pendulum task, all horizons converge to essentially the same asymptotic return, indicating that model error accumulation is negligible on this low-dimensional dynamics; longer imagination ($h = 15$) simply reaches the threshold slightly faster by providing more synthetic updates per real step. On IDP, a short horizon ($h = 1$) actually reaches the 9000 threshold nearly two times faster than $h = 3$, because for a balancing task the 1-step prediction

Env. (thr.)	Config.	Steps to Thr.	Asymptotic
Pendulum (-200)	MBPO-h1	5k (5/5)	-132.0 ± 10.2
	MBPO-h5	4k (5/5)	-133.1 ± 9.6
	MBPO-h10	4k (5/5)	-132.9 ± 9.3
	MBPO-h15	**3k (5/5)**	$\mathbf{-132.0 \pm 10.2}$
LunarLander (200)	**MBPO-h3**	**– (0/3)**	$\mathbf{-13.0 \pm 15.2}$
	MBPO-h5	– (0/3)	-31.6 ± 9.6
IDP (9000)	**MBPO-h1**	**18k (3/3)**	$\mathbf{9358.0 \pm 1.1}$
	MBPO-h3	35k (3/3)	9358.6 ± 0.7
HalfCheetah (3000)	**MBPO-h3**	**93k (3/3)**	$\mathbf{6297.5 \pm 1460.0}$
	MBPO-h5	53k (3/3)	3057.4 ± 2484.4

Table 3: MBPO ablation across horizon h. Best horizon per environment is **bolded**.

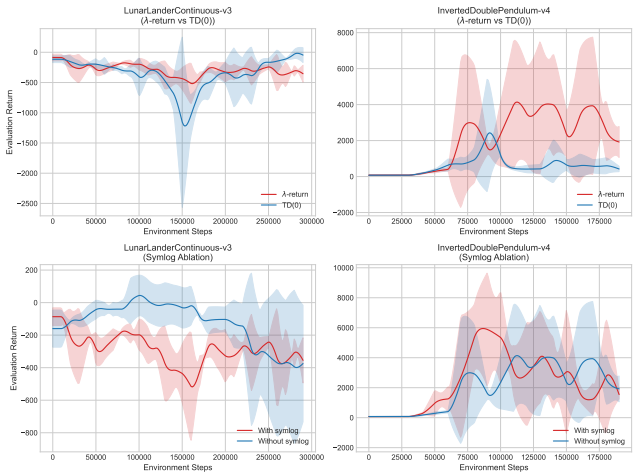

Figure 3: Dreamer architectural ablation on LunarLander and InvertedDoublePendulum (mean $\pm$ std). Top row compares λ-returns against a 1-step TD(0) target; bottom row toggles the symlog transformation.

is already informative and longer rollouts introduce unnecessary bias. The HalfCheetah result is the clearest illustration of the compounding-error phenomenon: $h = 3$ nearly doubles the asymptotic return of $h = 5$ (6297 vs. 3057) with much lower variance, showing that on a 17-dimensional locomotion task imagined trajectories beyond a few steps become unreliable and actively hurt policy learning. LunarLander is the negative case—neither $h = 3$ nor $h = 5$ crosses the 200 threshold within the budget, consistent with the hard-contact-dynamics analysis above.

Dreamer Architectural Ablation. Figure 3 compares our full Dreamer implementation against (i) replacing λ-returns with a 1-step TD(0) target and (ii) toggling the symlog transformation. Table 4 reports the quantitative summary.

Replacing the λ-return target with a 1-step TD(0) target consistently destabilizes actor learning. On LunarLander, the TD(0) variant shows very large mid-training dips (Figure 3, top-left). On IDP, the TD(0) run we have simultaneously disables symlog, so it is best read as a lower-

Env. (thr.)	Config.	Steps to Thr.	Asymptotic
LunarLander (200)	λ-ret + symlog (Base)	– (0/3)	-340.7 ± 84.7
	TD(0) replaces λ-ret	– (0/3)	-49.4 ± 67.2
	symlog removed	– (0/3)	-404.6 ± 376.5
IDP (9000)	λ-ret + symlog (Base)	70k (1/3)	2064.7 ± 2083.5
	symlog removed	– (0/3)	1857.3 ± 981.1
	TD(0) (also no symlog)†	– (0/3)	470.8 ± 297.4

Table 4: Dreamer ablation. "Base" denotes the full Dreamer configuration (λ-returns + symlog) reported in Table 2 for each environment. †On IDP our TD(0) run happened to also have symlog disabled, so it differs from Base on two axes; a dedicated TD(0)+symlog run was not executed. It is still informative as a "both components removed" lower bound.

bound "both components removed" ablation: the asymptotic return drops to 471 ± 297, roughly a quarter of Base. Taken together with the LunarLander evidence—where only λ-returns are toggled—this supports the argument in the Dreamer methodology section that multi-step imagined returns are critical for smoothing over the compounding errors of a learned world model when the critic is only a bootstrap anchor.

On IDP, removing symlog from Base causes all 3 seeds to stop reaching the threshold (Base: $1/3$ at 70k; no-symlog: $0/3$), and the asymptotic drops slightly from 2064 ± 2084 to 1857 ± 981. On LunarLander, neither the λ-ret+symlog Base nor the no-symlog variant reaches the threshold, but removing symlog makes the final return substantially more unstable (std 376 vs. 85) and lowers the mean. Overall, symlog is helpful for stability on environments with wide reward scales — and on IDP it is necessary for any seed to ever cross threshold — but in our low-dimensional state-vector setting with an MLP world model, neither λ-returns nor symlog is individually sufficient to reliably solve either environment, which matches the failure-case analysis about the fundamental limitation of MLP latent world models on tasks with discontinuous contact dynamics or very short episodes.

Conclusion

We have presented a controlled empirical comparison of three sample-efficiency paradigms—off-policy model-free RL (SAC), synthetic-data-augmentation model-based RL (MBPO), and latent world models (MLP Dreamer)—alongside an on-policy Stable-Baselines3 PPO baseline, on four continuous control tasks spanning a 3-dimensional Pendulum, an 8-dimensional LunarLander with hard-contact dynamics, an 11-dimensional InvertedDoublePendulum with short episodes, and a 17-dimensional HalfCheetah locomotion task. All results are reported as mean $\pm$ std over 3–5 independent seeds per (algorithm, environment) pair and summarized in Tables 2–4 and Figures 1–3.

Three conclusions emerge. First, SAC and MBPO are the most sample-efficient methods on the tasks where both succeed (Pendulum, IDP, and HalfCheetah), with MBPO marginally faster early due to its synthetic rollouts while SAC reaches the highest asymptotic return on HalfCheetah (7824 vs. MBPO's 6297); on LunarLander, however,

MBPO itself fails to cross the threshold ($0/3$ seeds, asymptotic -13 ± 15), so SAC is the only method in our suite that is robust across all four environments. Second, MBPO's advantage is horizon-sensitive: our ablation shows that on HalfCheetah, the asymptotic return drops from 6297 to 3057 when the imagination horizon is increased from $h = 3$ to $h = 5$, a direct empirical demonstration of compounding model error in high-dimensional state spaces. Third, and most informative, the MLP-encoder variant of Dreamer—though mathematically identical to the state-of-the-art latent world model on key components (RSSM, KL balancing with free nats, λ-returns with target critic, symlog transformation, and γ^τ-weighted actor loss over the imagined rollout)—fails to reliably solve either LunarLander or InvertedDoublePendulum (on IDP only $1/3$ seeds ever reach the threshold, and on LunarLander $0/3$). The architectural ablation confirms that λ-returns and symlog are both necessary but not sufficient.

Beyond the quantitative findings, this project teaches us how to approach counter-intuitive empirical results. Our initial hypothesis—shaped by the popular narrative around latent world models—was that Dreamer would dominate, on the assumption that a more elaborate model will yield stronger performance. When early experiments contradicted this expectation, our first reaction was not to accept the evidence and investigate the environment–architecture interaction, but to suspect implementation bugs and sweep hyperparameters in increasingly fine-grained ways, which consumed a substantial amount of compute and time. The turning point came only when we stepped back and analysed the failures from the perspective of the environments themselves (episode length, contact discontinuities) and of the inductive biases of the MLP RSSM, rather than blaming our implementation. In retrospect, we learned that (i) accepting that a method can legitimately under-perform on a given task—and searching for *why* it does so in terms of task structure and the method itself—is more valuable than an uncritical defense of the architecture; (ii) "more expressive model" is not a synonym for "better sample efficiency"; and (iii) when a hypothesis is not supported by the data, it is more productive to revisit the hypothesis than to keep retuning the system.

Future Work

Our current study is limited by its compute and interaction-step budget, and several natural follow-up experiments would strengthen the claims above.

More seeds for tighter confidence intervals. Three environments (LunarLander, IDP, HalfCheetah) are currently run with only 3 seeds per algorithm, which is at the low end of what is typically considered adequate in the RL literature. The large seed-to-seed variance we observe on some runs—most visibly SB3 PPO on IDP, where the asymptotic return is 6127 ± 4087—shows that 3 seeds are not always sufficient to sharply characterize an algorithm's behavior. As compute permits, we plan to bring every (algorithm, environment) pair to ≥ 5 seeds so that our Steps-to-Threshold and Asymptotic Return estimates have standard errors.

Disentangling short-episode failure of MLP Dreamer. In the *Failure Case* analysis above we attribute Dreamer's failure on IDP to the very short episode length interacting poorly with the RSSM burn-in. With more compute we would run the following controlled ablation on IDP: (i) train Dreamer with burn-in sequence length shrunk to $\{1, 2, 5, 10\}$ steps; (ii) train Dreamer with the recurrent state reset to zero at each imagination start (no burn-in at all); (iii) train a variant of the RSSM that replaces the GRU recurrence with a feed-forward or small Transformer encoder on a fixed context window. Comparing these variants would let us separate "burn-in window too long" (fixed by (i)) from "RNN cannot converge on short sequences" (would be fixed by (ii) or (iii) but not by (i)).

Beyond MLP encoders. All Dreamer experiments in this paper use a plain MLP observation encoder to match the state-vector dimensionality of our environments. Dreamer's historical strengths, however, come from richer encoders (CNNs for pixels, tokenizers for discrete observations). A natural follow-up is to revisit LunarLander and HalfCheetah with their pixel-based rendering modes and a CNN encoder, which would let us test whether Dreamer's under-performance here is specifically a consequence of the MLP encoder rather than of the algorithm as a whole.

Beyond preliminary thresholds. Our HalfCheetah "preliminary convergence threshold" of 3000 is deliberately chosen to measure early-stage sample efficiency under a restricted budget. Given a larger compute budget, we would also report time-to-*asymptotic* performance (e.g., the SAC paper's $15\,000$ reference after 3M steps), which would let us check whether MBPO's model-bias plateau at ~ 6300 eventually closes or is a true asymptotic ceiling.

Acknowledgements

We would like to thank Professor Christopher Amato for his insightful guidance throughout this project, which has been instrumental in shaping our analysis and interpretation of the results.

References

Chua, K.; Calandra, R.; McAllister, R.; and Levine, S. 2018. Deep Reinforcement Learning in a Handful of Trials using Probabilistic Dynamics Models. In *NeurIPS*.

Deisenroth, M. P.; and Rasmussen, C. E. 2011. PILCO: A Model-Based and Data-Efficient Approach to Policy Search. In *ICML*.

Fujimoto, S.; van Hoof, H.; and Meger, D. 2018. Addressing Function Approximation Error in Actor-Critic Methods. In *ICML*.

Ha, D.; and Schmidhuber, J. 2018. Recurrent World Models Facilitate Policy Evolution. In *NeurIPS*.

Haarnoja, T.; Zhou, A.; Abbeel, P.; and Levine, S. 2018a. Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor. In *ICML*.

Haarnoja, T.; Zhou, A.; Hartikainen, K.; Tucker, G.; Ha, S.; Tan, J.; Kumar, V.; Zhu, H.; Gupta, A.; Abbeel, P.; and

Levine, S. 2018b. Soft Actor-Critic Algorithms and Applications. *arXiv:1812.05905*.

Hafner, D.; Lillicrap, T.; Fischer, I.; Villegas, R.; Ha, D.; Lee, H.; and Davidson, J. 2019. Learning Latent Dynamics for Planning from Pixels. In *ICML*.

Hafner, D.; Lillicrap, T.; Ba, J.; and Norouzi, M. 2020. Dream to Control: Learning Behaviors by Latent Imagination. In *ICLR*.

Hafner, D.; Lillicrap, T.; Norouzi, M.; and Ba, J. 2021. Mastering Atari with Discrete World Models. In *ICLR*.

Hafner, D.; Pasukonis, J.; Ba, J.; and Lillicrap, T. 2023. Mastering Diverse Domains through World Models (DreamerV3). *arXiv:2301.04104*.

Higgins, I.; Matthey, L.; Pal, A.; Burgess, C.; Glorot, X.; Botvinick, M.; Mohamed, S.; and Lerchner, A. 2017. β-VAE: Learning Basic Visual Concepts with a Constrained Variational Framework. In *ICLR*.

Janner, M.; Fu, J.; Zhang, M.; and Levine, S. 2019. When to Trust Your Model: Model-Based Policy Optimization. In *NeurIPS*.

Kurutach, T.; Clavera, I.; Duan, Y.; Tamar, A.; and Abbeel, P. 2018. Model-Ensemble Trust-Region Policy Optimization. In *ICLR*.

Lillicrap, T. P.; Hunt, J. J.; Pritzel, A.; Heess, N.; Erez, T.; Tassa, Y.; Silver, D.; and Wierstra, D. 2016. Continuous Control with Deep Reinforcement Learning. In *ICLR*.

Mnih, V.; et al. 2015. Human-Level Control through Deep Reinforcement Learning. *Nature*, 518(7540): 529–533.

Schulman, J.; Wolski, F.; Dhariwal, P.; Radford, A.; and Klimov, O. 2017. Proximal Policy Optimization Algorithms. *arXiv:1707.06347*.

Sutton, R. S. 1990. Integrated Architectures for Learning, Planning, and Reacting Based on Approximating Dynamic Programming. In *ICML*.

Williams, R. J. 1992. Simple Statistical Gradient-Following Algorithms for Connectionist Reinforcement Learning. *Machine Learning*, 8(3–4): 229–256.